KYAMBOGO UNIVERSITY

FACULTY OF ENGINEERING

DEPARTMENT OF MECHATRONICS AND BIOMEDICAL ENGINEERING

BACHELOR OF MECHATRONICS AND BIOMEDICAL ENGINEERING

COURSE: COMPUTING II

TOPIC: TRANSPORT FARE MANAGEMENT SYSTEM REPORT.

GROUP 6 MEMBERS.

S/N	NAME	REGISTRATION NUMBER
1	KIBI DARIUS	25/U/BIO/01383/PD
2	OTIM AMOS	25/U/BIO/01416/PD
3	INNOCENT BRYAN VUNI	25/U/BIO/01372/PD
4	ODORA STEPHEN	25/U/BIO/01411/PD
5	KUSENA JIMMY	25/U/BIO/01389/PD
6	JAKIRA JUMA FAHAD	25/U/BIO/01374/PD

1. Introduction

This report documents the design and implementation of the Transport Fare Management System (TFMS), developed in the C programming language as part of the Group 6 Day Program C Programming assignment. The system simulates real-world passenger management for a taxi or bus service, enabling operators to register passengers, manage records, compute fares, and monitor vehicle status.

The program applies business rules including distance-based fares, passenger-type discounts, luggage surcharges, vehicle capacity limits, and departure readiness checks — all accessed through a secure, menu-driven console interface.

2. Problem Statement

Public transport operators in Uganda currently manage passenger records and fare calculations manually or with simple spreadsheets. This approach is error-prone, slow, and lacks enforcement of rules such as capacity limits and discount eligibility. The assignment required the development of a C program addressing the following needs:

- Register and store passenger records including name, type, distance, and luggage weight.
- Apply fare calculations based on distance travelled (UGX 500 per km base rate).
- Grant discounts for students (15%), children (20%), and elderly passengers (10%).
- Prevent boarding when the vehicle is at full capacity (30 seats).
- Add a surcharge of UGX 2,000 per kg for luggage exceeding 20 kg.
- Display 'not ready to depart' if the vehicle is less than 60% full.
- Report total revenue, the highest-fare passenger, and trip departure status.

3. System Features

3.1 Authentication

The program requires a valid username and password before granting access. Users are allowed a maximum of three attempts; after which the program exits automatically. Default credentials are username: admin and password: group6.

3.2 Passenger Registration

The operator enters a passenger's full name, type (Adult, Student, Child, Elderly), distance to destination (km), and luggage weight (kg). The system validates all inputs, enforces the 30-seat capacity limit, and automatically calculates and stores the fare.

3.3 Fare Calculation

Fares are computed using the following formula:

Base Fare = Distance (km) x UGX 500

Discounted = Base Fare x (1 - discount_rate)

Surcharge = MAX(0, luggage_kg - 20) x UGX 2,000

Final Fare = Discounted Fare + Surcharge

Passenger Type	**Discount**	**Example (100 km, 0 kg luggage)**
Adult	None (0%)	UGX 50,000
Student	15%	UGX 42,500
Child	20%	UGX 40,000
Elderly	10%	UGX 45,000

3.4 Search

Passengers can be found by either their unique system-generated ID or by a partial, case-insensitive name search. All matching records are displayed.

3.5 Update

Any field of a passenger record (name, type, distance, luggage weight) can be updated. The fare is automatically recalculated after any update.

3.6 Delete

A passenger record can be permanently removed after confirmation. The internal array is compacted to eliminate gaps.

3.7 Trip Summary

The trip summary displays: total revenue collected (UGX), the passenger who paid the highest fare, current fill level as a percentage, and whether the vehicle is ready to depart (requires at least 60% capacity).

4. Program Design and Structure

4.1 Data Structures

A struct named Passenger holds all data for each traveller. An enum PassengerType encodes the four passenger categories. An array of up to 50 Passenger records (and a global counter) serves as the in-memory database.

Field	Type	Description
id	int	Auto-incremented unique identifier starting at 1001
name	char[50]	Passenger full name
type	PassengerType	ADULT, STUDENT, CHILD, or ELDERLY
distance	double	Travel distance in kilometres
luggageWeight	double	Luggage mass in kilograms
fareCharged	double	Final fare in Ugandan Shillings (UGX)

4.2 Functions

The program is organized into well-named, single-purpose functions for readability and maintainability:

Function	Purpose
main()	Entry point; shows banner and calls authenticate() then mainMenu()
authenticate()	Validates credentials with up to 3 attempts
mainMenu()	Displays menu loop; dispatches to handler functions

registerPassenger()	Collects passenger data, validates, and stores record
searchPassenger()	Finds records by ID or partial name match
updatePassenger()	Modifies any field; recalculates fare
deletePassenger()	Removes a record after confirmation; compacts array
displayAllPassengers()	Tabular display of all registered passengers
tripSummary()	Revenue, highest fare, fill level, and depart status
calculateFare()	Applies discount, base rate, and luggage surcharge
findById()	Linear search returning array index or -1
typeName()	Returns string label for a PassengerType enum value
clearInputBuffer()	Flushes stdin to prevent stale-input bugs

5. Design Decisions and Justifications

- Array-based storage: An array of struct Passenger was chosen over dynamic linked lists to keep the implementation straightforward and within the scope of the course. The maximum of 50 records exceeds the 30-seat vehicle capacity, leaving room for future extension.
- Auto-incremented IDs: Passenger IDs start at 1001 and increment globally, ensuring uniqueness even after deletions and preventing accidental reuse of deleted IDs.
- Input buffer flushing: clearInputBuffer() is called after every scanf() to prevent newline characters from polluting subsequent fgets() calls — a common pitfall in C console programs.
- Case-insensitive name search: Both the search key and stored names are lowercased into temporary buffers before comparison, improving usability without modifying stored data.
- Fare recalculation on update: Whenever any field affecting the fare is changed, calculateFare() is called immediately so the stored fareCharged is always consistent with the current record.

- Separation of concerns: Each menu operation is encapsulated in its own function. calculateFare() is pure (no I/O), making it easily testable and reusable.

6. Challenges Encountered

6.1 Input Handling

Mixing scanf() and fgets() in C requires careful management of the newline character left in the input buffer. The group resolved this by implementing a dedicated clearInputBuffer() helper that reads and discards characters until a newline or EOF is encountered.

6.2 Capacity and Validation

Ensuring that no passenger is registered beyond the 30-seat capacity while also handling array indexing for deletions (array compaction) required careful off-by-one checking and thorough testing.

6.3 Discount Logic

Implementing the three discount categories with different rates while keeping the fare formula clean required the use of a switch statement within calculateFare(), which also cleanly handles the default (Adult) case with zero discount.

7. Conclusion

The Transport Fare Management System successfully meets all the requirements specified in the Group 6 Day assignment brief. The program provides secure authentication, full CRUD (Create, Read, Update, Delete) passenger record management, correct fare calculations with discounts and surcharges, vehicle capacity enforcement, and a comprehensive trip summary report.

The implementation demonstrates practical application of C programming concepts including structs, enums, arrays, functions, pointers, string handling, input validation, and decision-making logic. The codebase is organized for readability and can be extended to include file persistence, a graphical interface, or network connectivity in future iterations.

The full source code is available in the project GitHub repository along with this report and the presentation

The link to the GitHub repository is:

https://github.com/dariokibi806-dev/Transport-Fare-System